Decision trees, bagging and boosting

Gradient boosting, AdaBoost and XGBoost on twelve training runs; colours show where each number comes from

Udit Bhatia

ES 335 / ES 678 Machine Learning



Notation and colour code

One dataset (twelve past training runs) is used from start to finish. Every number is computed by hand, and **colours mark where a number came from**.

- **Part 1, trees:** questions are shaded by feature: rate batch size warm-up. Inside a gain, numbers are shaded by group: parent yes-group no-group.
- **Part 2, bagging:** every number read from a tree is shaded by that tree: tree 1 tree 2 tree 3.
- **Part 3, gradient boosting:** every number read from a stump is shaded by its question: rate batch warm-up.
- **Part 4, AdaBoost:** the same question colours, plus run weights and mistakes.
- **Part 5, XGBoost:** gains use the Part 1 colours (parent , yes , no); settings are orange , leaf weights teal .
- **Part 6:** comparison of the methods.

On tree figures, the **yellow path with thick arrows** is the route a new run follows.

Part 1: Decision trees

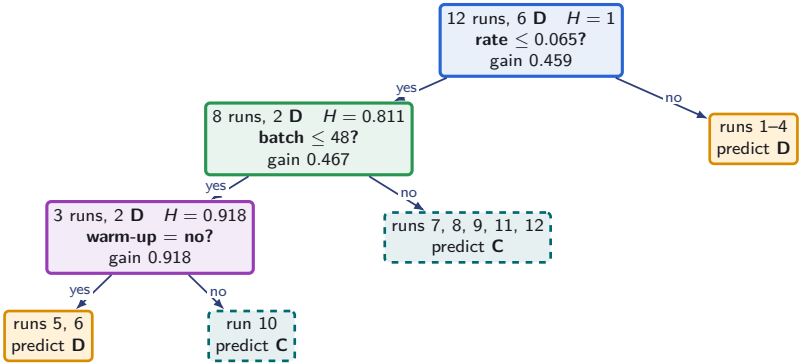
rate | batch size | warm-up | parent group | yes-group | no-group

The twelve runs

run	rate	batch	warm-up	out	run	rate	batch	warm-up	out
1	0.3	256	yes	**D**	7	0.03	256	no	**C**
2	0.3	32	no	**D**	8	0.03	64	yes	**C**
3	0.1	128	yes	**D**	9	0.01	128	yes	**C**
4	0.1	64	no	**D**	10	0.01	32	yes	**C**
5	0.03	32	no	**D**	11	0.001	64	no	**C**
6	0.01	32	no	**D**	12	0.001	256	yes	**C**

Each row is one past training run (mini-batch gradient descent). Features: **learning rate**, **batch size**, **warm-up**. Outcome: **D** diverged (loss blew up) or **C** converged. Six and six.

The decision tree grown from the twelve runs



A high learning rate diverges; otherwise a small batch without warm-up diverges; everything else converges. Yes goes left, no goes right.

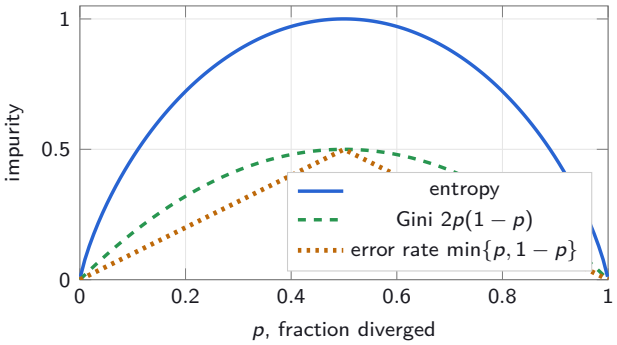
Entropy: how mixed is a group?

If a fraction p of the runs in a group diverged: $H(p) = -p \log_2 p - (1-p)\log_2(1-p)$.
$H = 0$ for a pure group, $H = 1$ for a 50/50 group.

group has	0 of 6 **D**	1 of 7	1 of 5	1 of 4	1 of 3	2 of 3	5 of 7	6 of 12
p	0	1/7	1/5	1/4	1/3	2/3	5/7	1/2
$H(p)$	0	0.5917	0.7219	0.8113	0.9183	0.9183	0.8631	1

Example: $H(\frac{1}{4}) = \frac{1}{4}(2) + \frac{3}{4}(0.4150) = 0.5 + 0.3113 = 0.8113$.

Three ways to measure mixing



All are 0 for a pure group and largest at $p = \frac{1}{2}$.

Entropy and Gini are **strictly concave**: a chord between two points lies below the curve. A split's weighted-average impurity is a point on such a chord, so a split never raises impurity and strictly lowers it unless both groups have the parent's p.

The error-rate tent is made of straight lines: splits can leave it unchanged even while making groups purer.

Scoring a question: the gain

A question splits n runs into a yes-group (n_Y runs) and a no-group (n_N runs).

$$\text{gain} = H(\text{parent}) - \left(\frac{n_Y}{n} H(\text{yes-group}) + \frac{n_N}{n} H(\text{no-group}) \right)$$

The weights stop a small pure group from looking as good as a large one.

Which thresholds? Sort the distinct values; cut at the midpoints.

feature	distinct values	thresholds tried
rate	0.001, 0.01, 0.03, 0.1, 0.3	0.0055, 0.02, 0.065, 0.2
batch size	32, 64, 128, 256	48, 96, 192
warm-up	no, yes	one question: warm-up = no

$4 + 3 + 1 = 8$ candidate questions at the start.

Your turn: choose the first question

Work on this, then compare with the next slide

Eight runs; **rate** is high or low, **warm-up** is yes or no.

run	1	2	3	4	5	6	7	8
rate	high	high	high	high	low	low	low	low
warm-up	no	no	yes	yes	no	no	yes	yes
out	**D**	**D**	**D**	**D**	**D**	**C**	**C**	**C**

1. Entropy of the whole group (5 **D**, 3 **C**).
2. Gain of the question **rate = high?**
3. Gain of the question **warm-up = no?**
4. Which question should the tree ask first?

Useful values: $H(\frac{5}{8}) = 0.9544$, $H(\frac{1}{4}) = H(\frac{3}{4}) = 0.8113$, $H(\frac{1}{2}) = 1$. (About 4 minutes.)

Answer: choose the first question

1. $H(\frac{5}{8}) =$ 0.9544 .

2. rate = high? Yes-group: runs 1–4 (4 runs, all **D**, $H = 0$). No-group: runs 5–8 (4 runs, 1 **D**, $H(\frac{1}{4}) = 0.8113$).

$$\text{gain} = 0.9544 - \left(\tfrac{4}{8} \times 0 + \tfrac{4}{8} \times 0.8113\right) = \mathbf{0.5488}$$

3. warm-up = no? Yes-group: runs 1, 2, 5, 6 (3 **D**, $H = 0.8113$). No-group: runs 3, 4, 7, 8 (2 **D**, $H = 1$).

$$\text{gain} = 0.9544 - \left(\tfrac{4}{8} \times 0.8113 + \tfrac{4}{8} \times 1\right) = \mathbf{0.0488}$$

4. Ask **rate = high?** first: it makes one group pure and the gain is more than ten times larger.

Follow-up. In the low-rate group (runs 5–8), warm-up gives gain $0.8113 - \frac{2}{4}(1) = 0.3113$, but runs 5 and 6 have identical features and different outcomes: no question can separate them, so the tree stops there with a leaf whose fraction of **D** is $\frac{1}{2}$.

Step 1: all eight candidate questions

question	yes-group	no-group	gain
rate ≤ 0.0055	2 runs, 0 **D**	10 runs, 6 **D**	0.1909
rate ≤ 0.02	5 runs, 1 **D**	7 runs, 5 **D**	0.1957
rate ≤ 0.065	8 runs, 2 **D**	4 runs, 4 **D**	**0.4591**
rate ≤ 0.2	10 runs, 4 **D**	2 runs, 2 **D**	0.1909
batch ≤ 48	4 runs, 3 **D**	8 runs, 3 **D**	0.0933
batch ≤ 96	7 runs, 4 **D**	5 runs, 2 **D**	0.0207
batch ≤ 192	9 runs, 5 **D**	3 runs, 1 **D**	0.0271
warm-up $=$ no	6 runs, 4 **D**	6 runs, 2 **D**	0.0817

Parent group: all 12 runs, 6 **D**, $H = 1$. Rate ≤ 0.065 wins: it makes runs 1–4 perfectly pure.

Step 1: gain of the best question

Rate ≤ 0.065. Left: runs 5–12 (8 runs, 2 **D**, $p = \frac{1}{4}$). Right: runs 1–4 (4 runs, all **D**, $H = 0$).

$$\text{gain} = 1 - \left(\tfrac{8}{12} \times 0.8113 + \tfrac{4}{12} \times 0\right) = 1 - 0.5409 = \mathbf{0.4591}$$

A runner-up: rate ≤ 0.02. Left: runs 6, 9, 10, 11, 12 (1 **D**). Right: runs 1–5, 7, 8 (5 **D**).

$$\text{gain} = 1 - \left(\tfrac{5}{12} \times 0.7219 + \tfrac{7}{12} \times 0.8631\right) = 1 - 0.3008 - 0.5035 = 0.1957$$

The 4-run group is pure: a leaf. The 8-run group continues.

Step 2: inside the 8-run group

question	yes-group	no-group	entropy gain	error-rate gain
rate ≤ 0.0055	2 runs, 0 **D**	6 runs, 2 **D**	0.1226	0
rate ≤ 0.02	5 runs, 1 **D**	3 runs, 1 **D**	0.0157	0
batch ≤ 48	3 runs, 2 **D**	5 runs, 0 **D**	**0.4669**	0.125
batch ≤ 96	5 runs, 2 **D**	3 runs, 0 **D**	0.2044	0
batch ≤ 192	6 runs, 2 **D**	2 runs, 0 **D**	0.1226	0
warm-up $=$ no	4 runs, 2 **D**	4 runs, 0 **D**	0.3113	0

Parent: runs 5–12, 2 **D**, $H = 0.8113$.

$$\text{gain}(\text{batch} \leq 48) = 0.8113 - \left(\tfrac{3}{8} \times 0.9183 + \tfrac{5}{8} \times 0\right) = 0.8113 - 0.3444 = \mathbf{0.4669}$$

Why error rate is a poor splitting criterion

Scoring by the drop in *error rate* (the fraction a majority vote gets wrong): five of the six questions score exactly 0, including warm-up, which isolates four pure **C** runs.

Why

In every one of those splits, both groups are still majority **C**, so a majority vote makes the same two mistakes (runs 5, 6) as before. The error rate sees no progress.

Entropy sees it

Entropy notices that the groups became purer, and ranks the questions sensibly. This is the straight-line part of the error-rate curve shown earlier.

Step 3: the last question, and when to stop

Group: runs 5, 6, 10 (2 **D**). All have batch 32, so no batch question exists.

question	yes	no	gain
rate ≤ 0.02	runs 6, 10	run 5	0.2516
warm-up = no	runs 5, 6	run 10	**0.9183**

gain $= 0.9183 - (\frac{2}{3}\,0 + \frac{1}{3}\,0) = 0.9183$: every leaf is pure, so we **stop**.

The algorithm

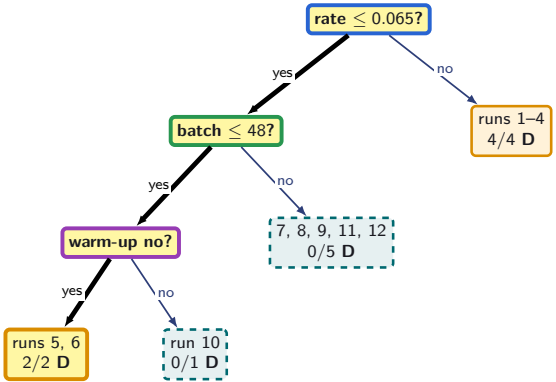
1. All runs in one group.
2. Group pure (or no valid question)? Make a leaf predicting the majority.
3. Else ask the question of largest gain.
4. Send runs yes/no; repeat inside each group.

Order? Each group uses only its own runs, so depth-first and breadth-first give the same tree; order matters only under a leaf budget.
Gini? Same three questions (gains 0.250, 0.208, 0.444).

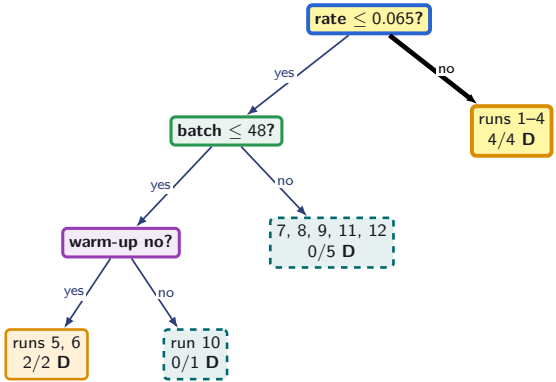
Predicting: the number read from the leaf

The tree reports the fraction of **D** among the training runs in the leaf (here 0 or 1).

Run A: rate 0.05, batch 32, warm-up no



Run B: rate 0.2, batch 64, warm-up yes



A: yes, yes, yes → runs 5, 6: 2/2 diverged, predict **D**. B: no at the root → runs 1–4: 4/4, predict **D**.

Your turn: predict with the tree

Work on this, then compare with the next slide

Use the tree grown from the twelve runs (previous slides). Predict each new run, and give the leaf it reaches.

new run	rate	batch	warm-up
P	0.05	128	no
Q	0.01	32	yes
R	0.5	16	no

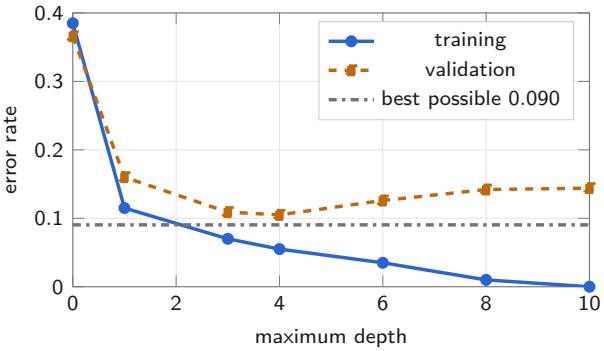
Which of the three predictions would you trust least, and why? (About 3 minutes.)

Answer: predict with the tree

run	path	prediction
P	rate ≤ 0.065 yes; batch $128 > 48$, no	leaf of runs 7, 8, 9, 11, 12: $0/5$ **D** $\rightarrow$ **C**
Q	rate ≤ 0.065 yes; batch ≤ 48 yes; warm-up is yes, so "warm-up $=$ no" is no	leaf of run 10: $0/1$ **D** $\rightarrow$ **C**
R	rate $0.5 > 0.065$, no	leaf of runs 1–4: $4/4$ **D** $\rightarrow$ **D**

Least trusted: **Q**. Its leaf holds a single training run, so the prediction rests on one observation; P and R rest on 5 and 4 runs. (The tree reports the same kind of answer for all three: it does not say how few runs support it.)

Depth and overfitting (simulated example)



200 training runs, 2000 validation runs, two features (simulated, not the twelve runs).

Training error only falls. Validation error bottoms out near depth 3–4, then rises: the deep tree memorises noise.

Depth is the complexity knob.

Pruning: charge a price per leaf

Keep the tree that minimises $\text{cost}_\alpha(T) = R(T) + \alpha|T|$ (R: training error rate, $|T|$: leaves). On the twelve runs:

tree	$\lvert T\rvert$	$R(T)$	$\alpha = 0.05$	0.1	0.2	0.4
full tree	4	0	**0.200**	0.400	0.800	1.600
warm-up merged	3	1/12	0.233	0.383	0.683	1.283
rate only	2	2/12	0.267	**0.367**	**0.567**	0.967
no question	1	6/12	0.550	0.600	0.700	**0.900**

Full beats rate-only when $4\alpha < \frac{2}{12} + 2\alpha$, i.e. $\alpha < \frac{1}{12}$. Choose α (or the depth) by cross-validation.

Instability of a single tree

Its first question can change

Re-draw the twelve runs with replacement (a *bootstrap sample*) and grow the tree again. In one such sample the first question is warm-up, not rate (gain 0.6175): the sample happened to miss the two **D** runs that used warm-up.

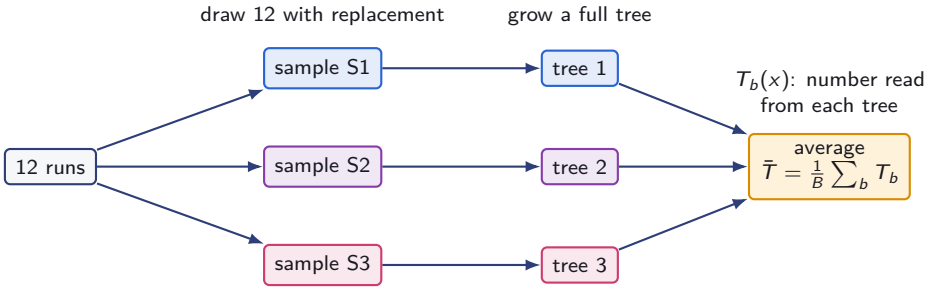
Consequence

A small change in the data changes the whole tree: **high variance**. Bagging averages this away; boosting attacks the opposite weakness (a small tree is too simple). Both next, on these same twelve runs.

Part 2: Bagging

tree 1 (sample S1) tree 2 (sample S2) tree 3 (sample S3)

Bagging: the procedure



Bagging mainly reduces **variance**: deep trees have low bias but change a lot with the data.

Step 1: draw three bootstrap samples

run	rate	batch	warm	out	S1	S2	S3
1	0.3	256	yes	**D**	3	0	0
2	0.3	32	no	**D**	0	2	0
3	0.1	128	yes	**D**	1	0	1
4	0.1	64	no	**D**	2	2	0
5	0.03	32	no	**D**	0	2	3
6	0.01	32	no	**D**	0	1	2
7	0.03	256	no	**C**	1	0	2
8	0.03	64	yes	**C**	2	1	2
9	0.01	128	yes	**C**	0	1	0
10	0.01	32	yes	**C**	1	2	0
11	0.001	64	no	**C**	2	1	0
12	0.001	256	yes	**C**	0	0	2
draws in the sample (row sums)					12	12	12

Each cell: how many times the sample drew that run (darker = more).

S1 missed runs 2, 5, 6, 9, 12.
S2 missed 1, 3, 7, 12.
S3 missed 1, 2, 4, 9, 10, 11.

A sample holds about 63% of the distinct runs: a run is missed with probability $(1 - \frac{1}{12})^{12} = 0.352$ ($\to 1/e = 0.368$).

Step 2: grow a tree on each sample (S1 and S2)

S1: 6 **D** draws (runs 1,1,1,3,4,4), 6 **C** draws (7,8,8,10,11,11), $H = 1$. **Rate** ≤ 0.065 splits them perfectly:

$$\text{gain} = 1 - \left(\tfrac{6}{12} \cdot 0 + \tfrac{6}{12} \cdot 0\right) = 1.0000 \quad \Rightarrow \text{ tree 1 stops after one question.}$$

S2: 7 **D** draws (2,2,4,4,5,5,6), 5 **C** draws (8,9,10,10,11), $H(\tfrac{7}{12}) = 0.9799$. Best root: **warm-up** = **no** (8 draws, 7 **D**, $H = 0.5436$; and 4 draws, all **C**):

$$\text{gain} = 0.9799 - \left(\tfrac{8}{12} \times 0.5436 + \tfrac{4}{12} \times 0\right) = 0.9799 - 0.3624 = 0.6175$$

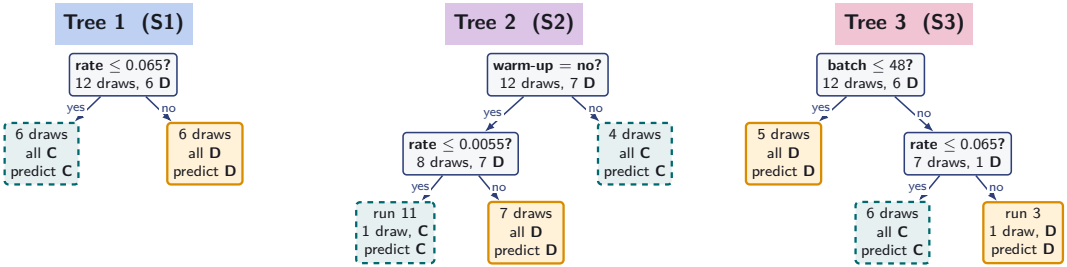
The 8-draw group holds run 11 (**C**), so tree 2 asks **rate** ≤ 0.0055 to isolate it (gain 0.5436).
S2 lacks runs 1 and 3 (the **D** runs *with* warm-up), so warm-up looks like the cause.

Step 2: tree 3, and the three trees

S3: 6 **D** draws (3,5,5,5,6,6), 6 **C** draws (7,7,8,8,12,12) , $H = 1$. Best root: **batch** ≤ 48 (5 draws all **D**; and 7 draws with 1 **D**, $H = 0.5917$):

$$\text{gain} = 1 - \left(\tfrac{5}{12} \times 0 + \tfrac{7}{12} \times 0.5917\right) = 1 - 0.3451 = 0.6549$$

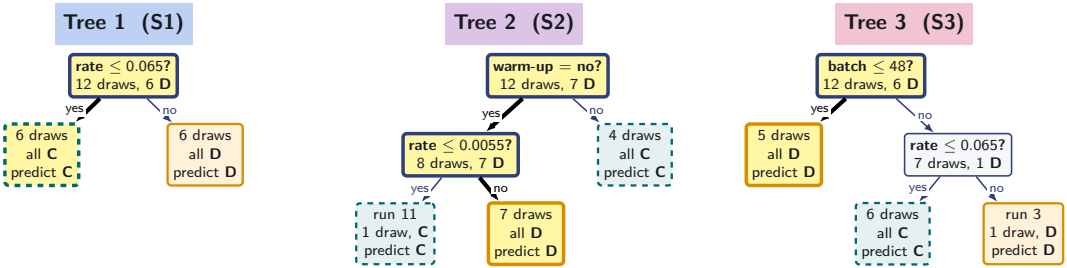
Then **rate** ≤ 0.065 isolates run 3.



All three use the learning rate, but at different positions and thresholds (0.0055 in tree 2).

Step 3: prediction of run A by each tree

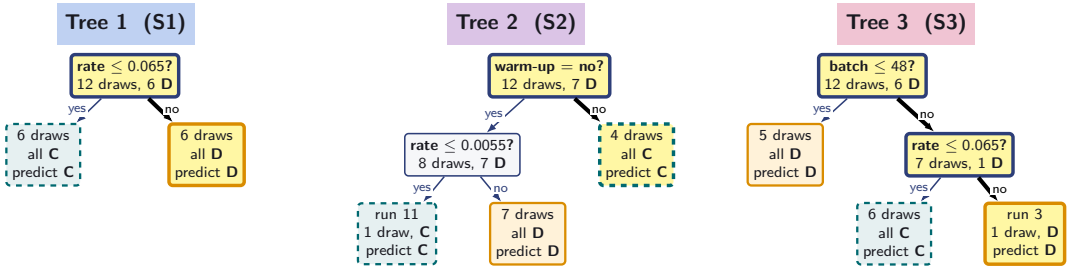
Run A: rate 0.05, batch 32, warm-up no



tree	leaf reached	fraction of **D** in it	$T_b(A)$
1	rate ≤ 0.065: yes	0/6	**0**
2	warm-up no; rate > 0.0055	7/7	**1**
3	batch ≤ 48: yes	5/5	**1**

Step 3: prediction of run B by each tree

Run B: rate 0.2, batch 64, warm-up yes



tree	leaf reached	fraction of **D** in it	$T_b(B)$
1	rate ≤ 0.065: no	6/6	**1**
2	warm-up yes (not "no")	0/4	**0**
3	batch > 48; rate > 0.065	1/1	**1**

Step 4: average the numbers, threshold at $\frac{1}{2}$

$\bar{T} = \frac{1}{B}\sum_b T_b$; the bagged answer is **D** if $\bar{T} > \frac{1}{2}$.

run A: $\bar{T} = \frac{1}{3}(0 + 1 + 1) = \frac{2}{3} \Rightarrow \mathbf{D}$ run B: $\bar{T} = \frac{1}{3}(1 + 0 + 1) = \frac{2}{3} \Rightarrow \mathbf{D}$

Same answer as the single tree, but the single tree showed no doubt; the vote shows one tree in three disagrees.

Three trees give a rough estimate

With 20,000 bootstrap trees (simulation), the fraction saying **D** settles at 0.872 at run A and 0.934 at run B. With 200 trees: 0.845 and 0.965.

Your turn: average the trees

Work on this, then compare with the next slide

For a new run, three bagged trees reach leaves containing these training draws:

	tree 1	tree 2	tree 3
draws in the leaf	4, of which 3 **D**	5, of which 0 **D**	2, of which 2 **D**

1. What number T_b does each tree give (the fraction of **D** in its leaf)?
2. Average them. What does the bagged model predict?
3. If each tree instead cast a hard vote (**D** or **C**), what is the share of votes for **D**?

(About 3 minutes.)

Answer: average the trees

1. $T_1 = \frac{3}{4} = 0.75,\ T_2 = 0,\ T_3 = 1.$

2. $\bar{T} = \frac{1}{3}(0.75 + 0 + 1) = 0.583 > \frac{1}{2}$: predict **D**.

3. Hard votes: tree 1 says **D**, tree 2 says **C**, tree 3 says **D**, so the share for **D** is $\frac{2}{3} = 0.667$. Both rules predict **D**, but with different confidence (0.583 against 0.667). Averaging leaf fractions is called soft voting; counting class votes is hard voting.

What σ^2 and ρ measure

Fix one new input, for example run A. Each bagged tree b gives a number T_b at A. To keep it simple, think of $T_b = 1$ if the tree says **D** and 0 if it says **C**.

- $\sigma^2 = \mathrm{Var}(T_b)$ is **how much one tree's answer at A varies from tree to tree**. It is 0 if every tree gives the same answer, and largest when the trees split evenly.
- ρ is **how much two different trees' answers move together**. $\rho = 0$: one tree's answer tells you nothing about another's. $\rho = 1$: the two trees always agree.
- $\mathrm{Cov}(T_b, T_c) = \rho\sigma^2$ combines the two.

share of trees saying **D** at the input, p	$\sigma^2 = p(1-p)$	meaning
1.00	0	every tree agrees: nothing to average away
0.872 (run A, simulation on the next slides)	0.1115	most trees agree, a few differ
0.50	0.25	trees split evenly: one tree is no better than a guess

σ^2 describes one tree at one input. Averaging B trees is useful because it shrinks this spread.

Why averaging helps: the variance (1 of 2)

Trees $T_1, \ldots, T_B$ at one input: each variance σ^2, every pair correlation ρ, so $\mathrm{Cov}(T_b, T_c) = \rho\sigma^2$. Goal: $\mathrm{Var}(\bar{T})$.

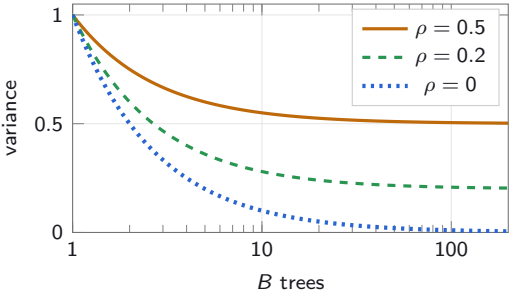
1. **Pull out the constant:** $\mathrm{Var}(\bar{T}) = \frac{1}{B^2}\mathrm{Var}\left(\sum_b T_b\right)$.
2. **Expand the sum:** $\mathrm{Var}\left(\sum_b T_b\right) = \sum_{b=1}^{B}\sum_{c=1}^{B}\mathrm{Cov}(T_b, T_c)$.
3. **Count the terms:** B with $b = c$ (each σ^2) and $B(B-1)$ with $b \neq c$ (each $\rho\sigma^2$): total $B\sigma^2 + B(B-1)\rho\sigma^2$.

Why averaging helps: the variance (2 of 2)

4. **Divide by B^2:**

$$\mathrm{Var}(\bar{T}) = \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2 = \boxed{\rho\sigma^2 + (1-\rho)\frac{\sigma^2}{B}}$$

5. **Read it:** $\rho = 0$: σ^2/B. $\rho = 1$: σ^2, no gain. $B \to \infty$: floor $\rho\sigma^2$.



$\sigma^2 = 1$: with $\rho = 0.5$, $B = 10$ gives 0.55, $B = 100$ gives 0.505.

Your turn: the variance of an average

Work on this, then compare with the next slide

One tree has variance $\sigma^2 = 0.8$ at a given input, and every pair of trees has correlation $\rho = 0.25$.

1. Variance of the average of $B = 16$ trees?
2. The floor as $B \to \infty$?
3. How many trees are needed to bring the variance within 0.01 of the floor?

Formula: $\text{Var}(\bar{T}) = \rho\sigma^2 + (1 - \rho)\sigma^2/B$. (About 3 minutes.)

Answer: the variance of an average

1. $0.25 \times 0.8 + 0.75 \times 0.8/16 = 0.2 + 0.0375 = \mathbf{0.2375}$.

2. Floor $= \rho\sigma^2 = \mathbf{0.2}$.

3. We need $(1-\rho)\sigma^2/B = 0.6/B \leq 0.01$, so $B \geq \mathbf{60}$ trees. Doubling to 120 trees lowers the variance only from 0.21 to 0.205: past this point more trees buy almost nothing, because the floor is set by ρ.

Checking the variance formula by simulation

Resample the twelve runs, grow a tree, record its prediction at run A, repeat. At A a tree says **D** with probability $p = 0.872$, so $\sigma^2_{\text{boot}} = p(1-p) = 0.1115$.

B trees averaged	measured variance of the average	σ^2_{boot}/B
1	0.1118	0.1115
3	0.0362	0.0372
10	0.0111	0.0111
50	0.0021	0.0022

They agree: this confirms the σ^2/B part for trees grown from the *same* twelve runs. A floor does not show here, because only resampling varies; the floor comes from the dataset itself varying.

Where the correlation ρ comes from

Given the data D, bootstrap trees are independent. Write $\mu(D) = E[T \mid D]$. For $b \neq c$:

$$\mathrm{Cov}(T_b, T_c) = \underbrace{E[\mathrm{Cov}(T_b, T_c \mid D)]}_{=0} + \mathrm{Var}(\mu(D)), \qquad \sigma^2 = \mathrm{Var}(\mu(D)) + E[\mathrm{Var}(T \mid D)]$$

$$\rho\sigma^2 = \mathrm{Var}(\mu(D)) \ \text{(part due to the data)}, \qquad (1-\rho)\sigma^2 = E[\mathrm{Var}(T \mid D)] \ \text{(part due to resampling)}$$

In words

Averaging removes the variation due to resampling but not the variation of the dataset itself: that is the floor. On a simulated 200-run example, $\sigma^2 = 0.071$, $\rho = 0.26$, floor 0.019; a single full tree's validation error 0.144 fell to 0.115 for a 200-tree bag.

Feature use across bagged trees; random forests

No: in plain bagging every split may use any feature. Root-question feature over 20,000 bootstrap trees:

	rate	batch	warm-up
bagging (all 3 features at every split)	80.8%	12.2%	6.9%
random forest, 2 random features per split	58.7%	28.0%	13.2%
random forest, 1 random feature per split	35.2%	33.2%	31.6%

Plain bagging starts with the learning rate in about four trees of five: trees alike, ρ high, floor high. A **random forest** shows each split only some features, so trees differ more, ρ falls, and so does the floor (simulated: ρ 0.26 $\rightarrow$ 0.20, floor 0.019 $\rightarrow$ 0.016), at the price of each tree being a little worse.

Out-of-bag error

run	true	trees not drawing it	frac. **D**	OOB	right?
1	**D**	63	0.746	**D**	yes
2	**D**	65	0.985	**D**	yes
3	**D**	68	0.632	**D**	yes
4	**D**	77	0.818	**D**	yes
5	**D**	70	0.557	**D**	yes
6	**D**	74	0.189	**C**	**no**
7	**C**	69	0.406	**C**	yes
8	**C**	66	0.455	**C**	yes
9	**C**	80	0.037	**C**	yes
10	**C**	69	0.783	**D**	**no**
11	**C**	70	0.514	**D**	**no**
12	**C**	68	0.000	**C**	yes

A run left out of a tree's sample is a fresh test run for that tree. With $B = 200$, each run was out of bag in about 70 trees.

OOB error: 3 wrong of 12 (0.25). The single tree scored on its own runs: 0 errors, an optimistic estimate.

No runs need to be held out.

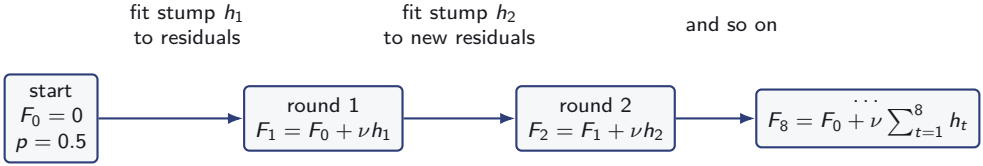
Bagging: summary of the procedure

step	what you do	which number is used
1. resample	draw 12 runs with replacement, B times	draw counts per run
2. grow	full tree on each sample	each tree's own questions and leaves
3. ask	send the new run down every tree	T_1, T_2, T_3: the **D**-fraction of the leaf reached
4. average	$\bar{T} = \frac{1}{B}\sum_b T_b$; **D** if $\bar{T} > \frac{1}{2}$	the numbers, added and divided by B
why it works	$\mathrm{Var}(\bar{T}) = \rho\sigma^2 + (1-\rho)\sigma^2/B$	σ^2: one tree; ρ: shared data
cannot fix	the floor $\rho\sigma^2$, and the bias	lower ρ: random forest

Part 3: Gradient boosting

rate stump | batch-size stump | warm-up stump

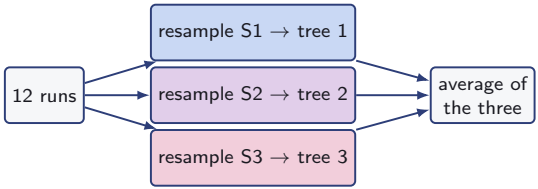
Boosting: the procedure



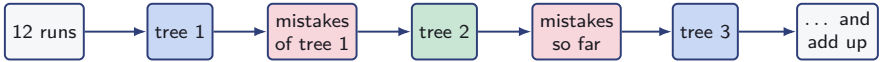
A **stump** has one question and two leaves. Boosting mainly reduces **bias**: each stump is too simple alone; their sum is not.

Boosting and bagging: the same twelve runs, used differently

Bagging: independent trees, averaged

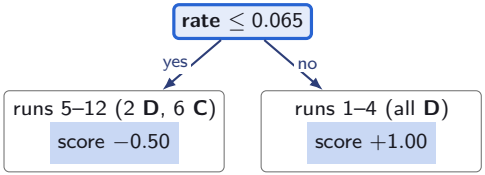


Boosting: each tree is grown on what the trees so far get wrong



In bagging the trees do not depend on each other. In boosting, **tree 2 cannot exist before tree 1**: it is grown on tree 1's mistakes. The next slides follow the same twelve runs, one tree at a time.

Boosting, step 1: tree 1 is one question



Boosting uses many **small** trees. Tree 1 is the first question of the tree in Part 1, rate ≤ 0.065, with two leaves.

Each leaf gives its runs a **score**: positive leans towards **D** (diverged), negative towards **C** (converged), 0 means undecided.

Before tree 1 every run has score 0. After it, runs 5–12 have score -0.50 and runs 1–4 have $+1.00$.

Where -0.50 and $+1.00$ come from is worked out after this example.

What tree 1 gets wrong

run	out	score	p	tree 1 says	error $y - p$	result
1	**D**	$+1.00$	0.731	**D**	$+0.269$	right
2	**D**	$+1.00$	0.731	**D**	$+0.269$	right
3	**D**	$+1.00$	0.731	**D**	$+0.269$	right
4	**D**	$+1.00$	0.731	**D**	$+0.269$	right
5	**D**	-0.50	0.378	**C**	$+0.622$	**wrong**
6	**D**	-0.50	0.378	**C**	$+0.622$	**wrong**
7	**C**	-0.50	0.378	**C**	-0.378	right
8	**C**	-0.50	0.378	**C**	-0.378	right
9	**C**	-0.50	0.378	**C**	-0.378	right
10	**C**	-0.50	0.378	**C**	-0.378	right
11	**C**	-0.50	0.378	**C**	-0.378	right
12	**C**	-0.50	0.378	**C**	-0.378	right

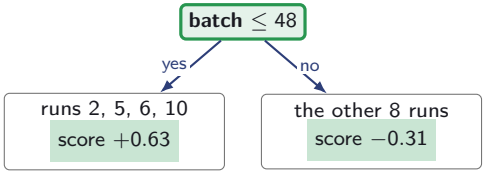
p turns a score into a probability of **D**; $y = 1$ for **D**, 0 for **C**. Tree 1 makes **2 mistakes in 12** (runs 5 and 6: diverged at a low rate). The error column is how far each prediction is from the outcome: runs 5, 6 are furthest off ($+0.62$).

Boosting, step 2: tree 2 is grown on the mistakes

Errors left by tree 1 ($y - p$):

runs	error
5, 6 (**D**, tree 1 says **C**)	$+0.62$
1–4 (**D**, right but unsure)	$+0.27$
7–12 (**C**, right)	-0.38

Tree 2 is not fitted to the outcomes again, but to **these errors**. It asks the question that separates large errors from small ones: batch ≤ 48 puts runs 2, 5, 6, 10 together (average error $+0.28$) against the other eight (average -0.14). That includes the two worst runs, 5 and 6.



Tree 2's scores push its group towards **D** ($+0.63$) or towards **C** (-0.31). They are **added** to the scores from tree 1.

Tree 2 changes the numbers

run	out	score, tree 1	tree 2 adds	score now	p	says	change
1	**D**	$+1.00$	-0.31	$+0.69$	0.667	**D**	
2	**D**	$+1.00$	$+0.63$	$+1.63$	0.836	**D**	
3	**D**	$+1.00$	-0.31	$+0.69$	0.667	**D**	
4	**D**	$+1.00$	-0.31	$+0.69$	0.667	**D**	
5	**D**	-0.50	$+0.63$	$+0.13$	0.532	**D**	fixed
6	**D**	-0.50	$+0.63$	$+0.13$	0.532	**D**	fixed
7	**C**	-0.50	-0.31	-0.81	0.309	**C**	
8	**C**	-0.50	-0.31	-0.81	0.309	**C**	
9	**C**	-0.50	-0.31	-0.81	0.309	**C**	
10	**C**	-0.50	$+0.63$	$+0.13$	0.532	**D**	**now wrong**
11	**C**	-0.50	-0.31	-0.81	0.309	**C**	
12	**C**	-0.50	-0.31	-0.81	0.309	**C**	

New score = score after tree 1 + what tree 2 adds. Runs 5 and 6 move from -0.50 to $+0.13$ ($p = 0.532$): now correct. Run 10 shares their leaf, is pushed up with them, and becomes a mistake: mistakes go from 2 to 1. Tree 3 is grown on the new errors.

How tree 2 differs from bagging

	bagging (Part 2)	boosting (this part)
tree 2 is grown on	a resample of the twelve runs; tree 1 plays no part	all twelve runs, fitted to the errors tree 1 left
tree 2 looks like	deep: grown until the leaves are pure	one question and two leaves
trees depend on each other	no: any order, or all at once	yes: tree 2 needs tree 1 first
numbers are combined by	averaging. Run 5: the three trees give 0 1 1, average 0.67	adding. Run 5: score -0.50 (tree 1) $+0.63$ (tree 2) $= +0.13$, and so on
each new tree reduces	the variance of the average	the bias: the remaining errors

Run 5 diverged. Bagging averages three independent opinions on it; boosting corrects the score step by step: after eight trees its probability of **D** is 0.74 (started at 0.5, 0.378 after tree 1).

The math behind the scores: definitions

Tree 1 and tree 2 used these quantities; here they are defined.

- **Score** F: any real number (the log-odds of **D**); every run starts at $F = 0$.
- **Probability** $p = S(F) = \dfrac{1}{1 + e^{-F}}$: turns a score into a number between 0 and 1. $S(0) = 0.5$, $S(1) = 0.731$, $S(-0.5) = 0.378$.
- **Outcome** $y = 1$ if **D**, 0 if **C**.
- **Residual** $r = y - p$: the error column in the table of tree 1's mistakes.
- **Log loss** $L = -y \ln p - (1 - y) \ln(1 - p)$: small when p is close to the outcome.

Example, run 5 after tree 1: $F = -0.50$, $p = 0.378$, $y = 1$, so $r = 1 - 0.378 = 0.622$.

Why the next tree is fitted to the residuals

Treat each run's score as a number we can change. How does one run's loss change with its own score? Using $S'(F) = p(1-p)$,

$$\frac{dL}{dF} = \Big(-\frac{y}{p} + \frac{1-y}{1-p}\Big)p(1-p) = -y(1-p) + (1-y)p = p - y = -r.$$

So the residual is the *negative gradient* of the loss: moving each score in the direction of its residual lowers the loss, which is gradient descent on the log loss.

A stump cannot move all twelve scores freely. It can only add one number to each of two groups. Tree 2 chose the question whose two groups have the most different average residual, because that is the best two-number correction.

The value assigned to a leaf

Curvature. $\frac{d^2L}{dF^2} = \frac{d}{dF}(p - y) = p(1 - p)$: largest (0.25) at $p = 0.5$, small when the prediction is confident.

Add the same number γ to the scores of all runs in a leaf. To second order the loss changes by $\gamma \sum(p_i - y_i) + \frac{1}{2}\gamma^2 \sum p_i(1 - p_i)$. Setting the derivative with respect to γ to zero:

$$\boxed{\gamma = \frac{\sum_{\text{leaf}} r_i}{\sum_{\text{leaf}} p_i(1 - p_i)}} \quad \text{(gradient over curvature: a Newton step)}$$

Then the score of each run in the leaf is increased by $\nu\gamma$, where $\nu = 0.5$ is the step size. In round 1 the leaves have $\gamma = -1.00$ and $+2.00$, so the scores above were $\nu\gamma = -0.50$ and $+1.00$.

One round of boosting, in four steps

(a) $p_i = S(F_i)$ and $r_i = y_i - p_i$ for every run.

(b) Score each possible question by $n_L \bar{r}_L^2 + n_R \bar{r}_R^2$ (how much the squared error falls if each group gets its own residual value); keep the question with the largest score.

(c) Leaf values $\gamma = \sum r / \sum p(1-p)$ for the two leaves.

(d) Update $F_i \leftarrow F_i + \nu \gamma_{\text{leaf}(i)}$, with step size $\nu = 0.5$.

The next slides carry out steps (a) to (d) for round 1 and then for round 2.

Round 1: choose the question

All scores $0 \Rightarrow p = 0.5$, $r = +0.5$ for **D**, -0.5 for **C**. Candidate scores $n_L \bar{r}_L^2 + n_R \bar{r}_R^2$:

question	score
rate ≤ 0.065	1.500
rate ≤ 0.02	0.771
rate ≤ 0.0055	0.600

Winner **rate ≤ 0.065** . Yes: runs 5–12, sum of $r = -2$. No: runs 1–4, sum $+2$. Every $p(1-p) = 0.25$:

$$\gamma_{\text{yes}} = \frac{-2}{8 \times 0.25} = -1.000\,, \qquad \gamma_{\text{no}} = \frac{+2}{4 \times 0.25} = +2.000$$

Round 1: update every run

run	out	p before	r	leaf	γ	$\nu\gamma$	F after	p after
1	**D**	0.500	+0.500	no	+2.000	+1.000	+1.000	0.731
2	**D**	0.500	+0.500	no	+2.000	+1.000	+1.000	0.731
3	**D**	0.500	+0.500	no	+2.000	+1.000	+1.000	0.731
4	**D**	0.500	+0.500	no	+2.000	+1.000	+1.000	0.731
5	**D**	0.500	+0.500	yes	-1.000	-0.500	-0.500	0.378
6	**D**	0.500	+0.500	yes	-1.000	-0.500	-0.500	0.378
7	**C**	0.500	-0.500	yes	-1.000	-0.500	-0.500	0.378
8	**C**	0.500	-0.500	yes	-1.000	-0.500	-0.500	0.378
9	**C**	0.500	-0.500	yes	-1.000	-0.500	-0.500	0.378
10	**C**	0.500	-0.500	yes	-1.000	-0.500	-0.500	0.378
11	**C**	0.500	-0.500	yes	-1.000	-0.500	-0.500	0.378
12	**C**	0.500	-0.500	yes	-1.000	-0.500	-0.500	0.378

Shaded: the two numbers read from this stump. Log loss $0.693 \to 0.504$. Runs 5, 6 still wrong ($p = 0.378$).

Round 2: choose the question

Residuals now: runs 5, 6: 0.622 (most wrong); runs 1–4: 0.269; runs 7–12: -0.378. Scores: batch ≤ 48 0.469; rate ≤ 0.065 0.419; rate ≤ 0.0055 0.351; warm-up 0.334.

Winner batch ≤ 48. Yes: runs 2, 5, 6, 10. No: the other 8.

$$\gamma_{\text{yes}} = \frac{1.136}{0.902} = +1.260, \qquad \gamma_{\text{no}} = \frac{-1.081}{1.765} = -0.612$$

Yes-group: residual sum $0.269 + 0.622 + 0.622 - 0.378$; curvature $0.731 \times 0.269 + 3 \times 0.378 \times 0.622$.

Your turn: one leaf's update in boosting

Work on this, then compare with the next slide

After round 1 every run in $\{1, 3, 4\}$ has $F = 1.0$ and diverged ($y = 1$). Suppose a stump places exactly these three runs in one leaf. Use $\nu = 0.5$.

1. p and the residual $r = y - p$ of each run.
2. $\gamma = \sum r / \sum p(1 - p)$ for the leaf.
3. The update $\nu\gamma$, the new score F, and the new probability p.
4. Repeat for a leaf holding runs 5 and 6 ($F = -0.5$, also $y = 1$). Which leaf receives the larger γ, and why?

Useful: $S(1) = 0.731$, $S(-0.5) = 0.378$. (About 5 minutes.)

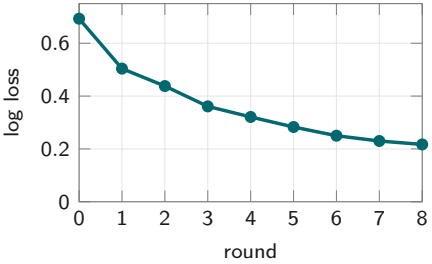
Answer: one leaf's update in boosting

	leaf $\{1, 3, 4\}$	leaf $\{5, 6\}$
p	0.731	0.378
$r = y - p$	0.269 each	0.622 each
$p(1-p)$	0.197 each	0.235 each
$\gamma = \sum r / \sum p(1-p)$	$0.807/0.590 = \mathbf{1.368}$	$1.245/0.470 = \mathbf{2.649}$
$\nu\gamma$	$+0.684$	$+1.324$
new F	1.684	0.824
new p	0.843	0.695

4. Runs 5 and 6 get the larger step: they are the runs the model is most wrong about. For a leaf in which all runs have the same p and $y = 1$, $\gamma = 1/p$ exactly, so the smaller the probability given to the correct outcome, the bigger the push.

All eight rounds

round	question	γ yes	γ no	loss	wrong
0	(all scores 0)			0.693	6
1	rate ≤ 0.065	-1.000	+2.000	0.504	2 (runs 5, 6)
2	batch ≤ 48	+1.260	-0.612	0.438	1 (runs 10)
3	rate ≤ 0.065	-0.629	+1.448	0.361	2 (runs 5, 6)
4	warm-up $=$ no	+0.800	-0.717	0.321	0
5	rate ≤ 0.065	-0.423	+1.279	0.283	0
6	batch ≤ 48	+0.949	-0.568	0.250	0
7	rate ≤ 0.02	-0.597	+0.577	0.230	0
8	batch ≤ 48	+0.603	-0.426	0.217	0



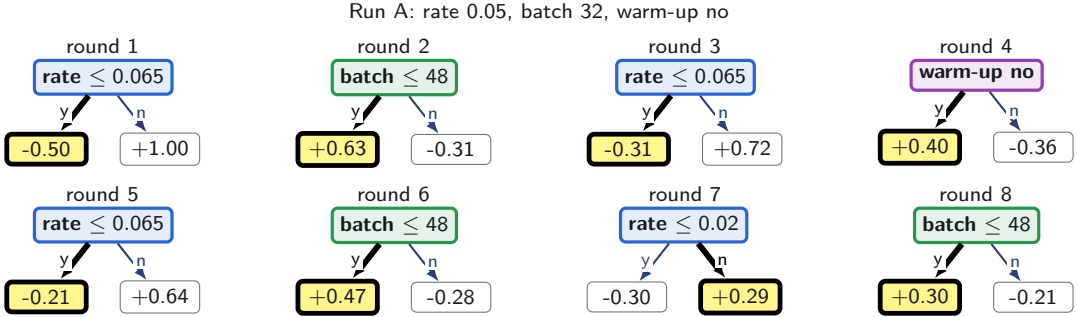
Loss falls every round; wrong runs go $6, 2, 1, 2, 0$. The single tree's three questions (rate, batch, warm-up) are rounds 1, 2 and 4.

Contribution of each stump to each run's score

run	out	r1 rate	r2 batch	r3 rate	r4 warm	r5 rate	r6 batch	r7 rate	r8 batch	F_8	p
1	**D**	+1.000	-0.306	+0.724	-0.358	+0.639	-0.284	+0.289	-0.213	+1.490	0.816
2	**D**	+1.000	+0.630	+0.724	+0.400	+0.639	+0.474	+0.289	+0.302	+4.458	0.989
3	**D**	+1.000	-0.306	+0.724	-0.358	+0.639	-0.284	+0.289	-0.213	+1.490	0.816
4	**D**	+1.000	-0.306	+0.724	+0.400	+0.639	-0.284	+0.289	-0.213	+2.248	0.905
5	**D**	-0.500	+0.630	-0.315	+0.400	-0.212	+0.474	+0.289	+0.302	+1.069	0.744
6	**D**	-0.500	+0.630	-0.315	+0.400	-0.212	+0.474	-0.298	+0.302	+0.482	0.618
7	**C**	-0.500	-0.306	-0.315	+0.400	-0.212	-0.284	+0.289	-0.213	-1.141	0.242
8	**C**	-0.500	-0.306	-0.315	-0.358	-0.212	-0.284	+0.289	-0.213	-1.899	0.130
9	**C**	-0.500	-0.306	-0.315	-0.358	-0.212	-0.284	-0.298	-0.213	-2.486	0.077
10	**C**	-0.500	+0.630	-0.315	-0.358	-0.212	+0.474	-0.298	+0.302	-0.277	0.431
11	**C**	-0.500	-0.306	-0.315	+0.400	-0.212	-0.284	-0.298	-0.213	-1.728	0.151
12	**C**	-0.500	-0.306	-0.315	-0.358	-0.212	-0.284	-0.298	-0.213	-2.486	0.077
A	?	-0.500	+0.630	-0.315	+0.400	-0.212	+0.474	+0.289	+0.302	+1.069	0.744
B	?	+1.000	-0.306	+0.724	-0.358	+0.639	-0.284	+0.289	-0.213	+1.490	0.816

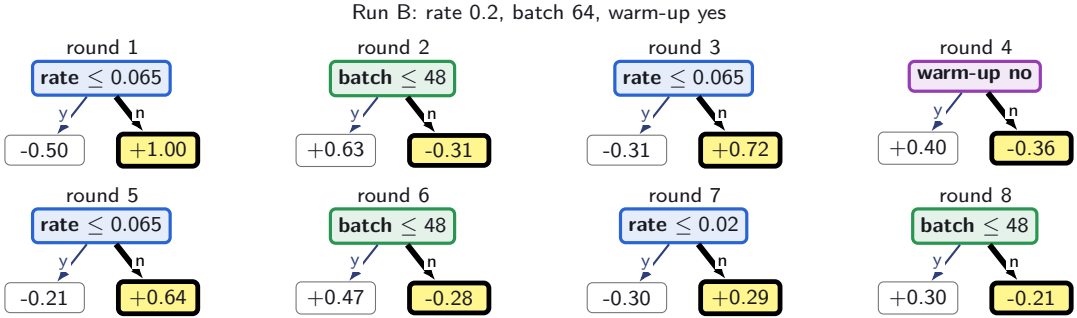
Entry $= \nu\gamma$ of the leaf the run reaches in that round's stump. F_8 is the row sum, $p = S(F_8)$. Runs A, B: $F_8 = 1.069, 1.490$; $p = 0.744, 0.816$.

Run A: the leaf reached in each stump



Yellow leaf = where A ends; its number is the entry in the table of contributions. The eight numbers add to $F_8 = 1.069$.

Run B: the leaf reached in each stump



The eight numbers add to $F_8 = 1.490$. All three rate stumps (1, 3, 5) push strongly towards **D**; batch and warm-up push back a little.

Step size and stopping

step size ν	first round with no wrong run	loss after 8 rounds	rounds to reach loss 0.217
1.0	4	0.087	4 (loss 0.196)
0.5	4	0.217	8
0.1	22	0.473	44 (loss 0.215)

A smaller step trusts each stump less: more rounds, each a safer correction. The loss keeps falling even after training error is 0: **training data can never tell us when to stop**.

Boosting can overfit

Simulated (200 runs, depth-3 trees, $\nu = 0.1$): training error 0 by round 60, held-out error best near round 66 (0.108), then rising to 0.137 by round 1000. Stop on held-out error.

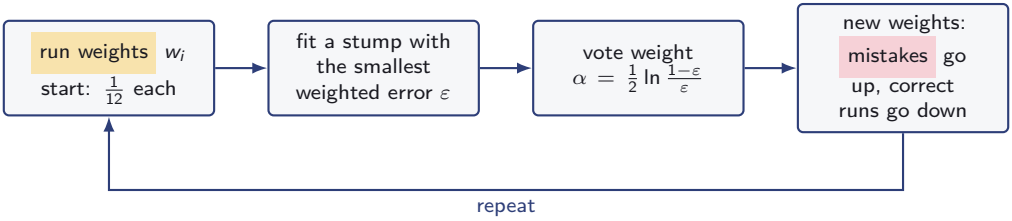
Boosting: summary of the procedure

step	what you do	which number is used
start	$F_i = 0$, so $p_i = 0.5$	none
(a) residuals	$r_i = y_i - p_i$	each run's current p
(b) question	largest $n_L \bar{r}_L^2 + n_R \bar{r}_R^2$	the residuals of the two groups
(c) leaf values	$\gamma = \sum r / \sum p(1-p)$	γ yes leaf , γ no leaf
(d) update	$F_i \leftarrow F_i + \nu\gamma$	$\nu\gamma$ of the leaf reached
predict	$F = \nu \sum_t \gamma_t$(leaf in stump t), $p = S(F)$	one number per stump (the table of contributions)
stop	when held-out error stops falling	–

Part 4: AdaBoost

run weight mistake and each stump's numbers shaded by its question: rate batch warm-up

AdaBoost: the procedure



Gradient boosting (Part 3) fits each stump to the residuals. **AdaBoost** keeps the original labels and instead *changes how much each run counts*: a run that the stumps so far get wrong counts more in the next round. The final model is a weighted vote of the stumps.

AdaBoost: definitions

- **Labels.** $y_i = +1$ if the run diverged (**D**), -1 if it converged (**C**). A stump gives $h(x) = \pm 1$: each leaf predicts the class with the larger total weight in that leaf.
- **Run weights** $w_i \geq 0$ with $\sum_i w_i = 1$; at the start $w_i = \frac{1}{12}$.
- **Weighted error** of a stump: $\varepsilon = \sum_{i:\, h(x_i) \neq y_i} w_i$ (the total weight of its mistakes).
- **Vote weight** $\alpha = \frac{1}{2} \ln \dfrac{1-\varepsilon}{\varepsilon}$: large when ε is small, 0 when $\varepsilon = \frac{1}{2}$, negative when $\varepsilon > \frac{1}{2}$.
- **Weight update.** $w_i \leftarrow \dfrac{w_i \, e^{-\alpha \, y_i \, h(x_i)}}{Z}$, so a correct run is multiplied by $e^{-\alpha}$ and a mistake by $e^{+\alpha}$; Z makes the weights add to 1.
- **Model after T rounds.** $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$; predict **D** if $F(x) > 0$, **C** otherwise.

Why this vote weight

Multiplying correct runs by $e^{-\alpha}$ and mistakes by $e^{+\alpha}$ gives total weight

$$Z(\alpha) = (1-\varepsilon)e^{-\alpha} + \varepsilon e^{\alpha}.$$

Choose α to make Z as small as possible: $\dfrac{dZ}{d\alpha} = -(1-\varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0 \Rightarrow e^{2\alpha} = \dfrac{1-\varepsilon}{\varepsilon}$, so

$$\boxed{\alpha = \tfrac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}}, \qquad Z = 2\sqrt{\varepsilon(1-\varepsilon)}.$$

What the update does. After normalising, the mistakes carry total weight $\dfrac{\varepsilon e^{\alpha}}{Z} = \dfrac{\varepsilon\sqrt{(1-\varepsilon)/\varepsilon}}{2\sqrt{\varepsilon(1-\varepsilon)}} = \tfrac{1}{2}$ and the correct runs carry the other $\tfrac{1}{2}$.

So the stump just used has weighted error exactly $\tfrac{1}{2}$ on the new weights: it is no better than a random guess, so the next stump must find something different. The loss being reduced is the exponential loss e^{-yF} (a later slide shows this).

Your turn: the vote weight

Work on this, then compare with the next slide

Compute $\alpha = \frac{1}{2} \ln \frac{1-\varepsilon}{\varepsilon}$ for four stumps with weighted error

ε	0.1	0.3	0.5	0.7
α				

Then answer: which stump gets the largest vote, and what does the vote of the last stump mean? (About 2 minutes.)
Useful: $\ln 9 = 2.197$, $\ln \frac{7}{3} = 0.847$.

Answer: the vote weight

ε	0.1	0.3	0.5	0.7
α	$\frac{1}{2}\ln 9 = 1.099$	$\frac{1}{2}\ln \frac{7}{3} = 0.424$	$\frac{1}{2}\ln 1 = 0$	$\frac{1}{2}\ln \frac{3}{7} = -0.424$

The stump with $\varepsilon = 0.1$ gets the largest vote. A stump at $\varepsilon = 0.5$ gets no vote at all. A stump with $\varepsilon = 0.7$ is wrong more often than not, so its vote is *negative*: AdaBoost uses it by flipping its answer. (If $\varepsilon \to 0$ then $\alpha \to \infty$: a perfect stump would take over the vote, so implementations cap α.)

Round 1: choose the stump

All twelve weights are $\frac{1}{12}$, so the weighted error is (number of mistakes)/12. Best five questions:

question	mistakes	ε
rate ≤ 0.065	2	0.1667
rate ≤ 0.02	3	0.2500
rate ≤ 0.0055	4	0.3333
rate ≤ 0.2	4	0.3333
batch ≤ 48	4	0.3333

Winner **rate ≤ 0.065**. Yes-group (runs 5–12): 6 **C**, 2 **D**, so it predicts **C**. No-group (runs 1–4): all **D**, so it predicts **D**. Mistakes: runs 5 and 6 (diverged, but low rate).

$$\varepsilon_1 = w_5 + w_6 = \frac{2}{12} = 0.1667, \qquad \alpha_1 = \frac{1}{2}\ln\frac{1-0.1667}{0.1667} = \frac{1}{2}\ln 5 = 0.8047$$

Round 1: update the weights

run	y	$h_1(x)$	correct?	w before	multiplier	$w\times$ mult.	w after
1	**D**	**D**	yes	0.0833	0.447	0.0373	0.0500
2	**D**	**D**	yes	0.0833	0.447	0.0373	0.0500
3	**D**	**D**	yes	0.0833	0.447	0.0373	0.0500
4	**D**	**D**	yes	0.0833	0.447	0.0373	0.0500
5	**D**	**C**	no	0.0833	2.236	0.1863	0.2500
6	**D**	**C**	no	0.0833	2.236	0.1863	0.2500
7	**C**	**C**	yes	0.0833	0.447	0.0373	0.0500
8	**C**	**C**	yes	0.0833	0.447	0.0373	0.0500
9	**C**	**C**	yes	0.0833	0.447	0.0373	0.0500
10	**C**	**C**	yes	0.0833	0.447	0.0373	0.0500
11	**C**	**C**	yes	0.0833	0.447	0.0373	0.0500
12	**C**	**C**	yes	0.0833	0.447	0.0373	0.0500

$e^{-\alpha_1} = 0.447$ for a correct run, $e^{+\alpha_1} = 2.236$ for a mistake. $Z = \sum = 0.7454$; dividing by Z: 0.25 for each mistake, 0.05 for every other run.

Your turn: the second stump

Work on this, then compare with the next slide

Weights after round 1: runs 5 and 6 have $w = 0.25$; every other run has $w = 0.05$.

Candidate stump batch ≤ 48: yes-group (runs 2, 5, 6, 10) predicts **D**; no-group (runs 1, 3, 4, 7, 8, 9, 11, 12) predicts **C**.

1. Which runs does it get wrong?
2. Its weighted error ε? Its ordinary error rate (mistakes/12)?
3. Its vote weight α?
4. Suppose the round-1 stump (rate ≤ 0.065) were used again on these weights: its ε and α?

Useful: $\ln 4 = 1.386$. (About 4 minutes.)

Answer: the second stump

1. Run 10 (**C**, predicted **D**) and runs 1, 3, 4 (**D**, predicted **C**).

2. $\varepsilon = 4 \times 0.05 =$ 0.200 . Ordinary error rate $\frac{4}{12} = 0.333$: the weighted error is smaller because the mistakes are runs that already count little.

3. $\alpha = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4 =$ 0.693 .

4. The round-1 stump errs on runs 5 and 6: $\varepsilon = 0.25 + 0.25 = 0.5$, so $\alpha = 0$. Reusing it adds nothing, which is the result of the previous slide: after an update, the stump just used has weighted error $\frac{1}{2}$.

Tie. batch ≤ 48 , warm-up $=$ no and rate ≤ 0.0055 all have $\varepsilon = 0.2$. Rule used in this deck: prefer a feature not asked before, then the order rate, batch, warm-up. So round 2 asks batch.

Round 2: update the weights

run	y	$h_2(x)$	correct?	w before	multiplier	$w\times$ mult.	w after
1	**D**	**C**	no	0.0500	2.000	0.1000	0.1250
2	**D**	**D**	yes	0.0500	0.500	0.0250	0.0312
3	**D**	**C**	no	0.0500	2.000	0.1000	0.1250
4	**D**	**C**	no	0.0500	2.000	0.1000	0.1250
5	**D**	**D**	yes	0.2500	0.500	0.1250	0.1563
6	**D**	**D**	yes	0.2500	0.500	0.1250	0.1563
7	**C**	**C**	yes	0.0500	0.500	0.0250	0.0312
8	**C**	**C**	yes	0.0500	0.500	0.0250	0.0312
9	**C**	**C**	yes	0.0500	0.500	0.0250	0.0312
10	**C**	**D**	no	0.0500	2.000	0.1000	0.1250
11	**C**	**C**	yes	0.0500	0.500	0.0250	0.0312
12	**C**	**C**	yes	0.0500	0.500	0.0250	0.0312

$\alpha_2 = 0.693$: multipliers 0.500 (correct), 2.000 (mistake); $Z = 0.8000$. Runs 1, 3, 4, 10 carry 0.125 each; runs 5, 6 fall to 0.156.

Five rounds

round	stump	yes, no predict	ε	α	stump's mistakes	model's mistakes	$\overline{e^{-yF}}$
1	rate ≤ 0.065	**C**, **D**	0.1667	0.805	5, 6	5, 6	0.745
2	batch ≤ 48	**D**, **C**	0.2000	0.693	1, 3, 4, 10	5, 6	0.596
3	rate ≤ 0.0055	**C**, **D**	0.2188	0.636	7, 8, 9, 10	10	0.493
4	rate ≤ 0.065	**C**, **D**	0.2000	0.693	5, 6	5, 6	0.394
5	warm-up $=$ no	**D**, **C**	0.1571	0.840	1, 3, 7, 11	none	0.287

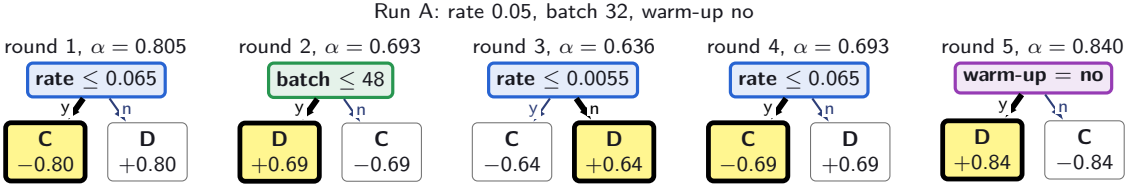
The model's mistakes use all stumps so far ($F = \sum \alpha_t h_t$). The *count* of mistakes can go up (round 4) but the exponential loss $\overline{e^{-yF}}$ (mean over the 12 runs) falls every round. After round 5 the model classifies all twelve runs correctly.

Contribution of each stump to each run's vote

run	out	r1 rate	r2 batch	r3 rate	r4 rate	r5 warm	F_5	predicts
1	**D**	+0.805	−0.693	+0.636	+0.693	−0.840	+0.601	**D**
2	**D**	+0.805	+0.693	+0.636	+0.693	+0.840	+3.667	**D**
3	**D**	+0.805	−0.693	+0.636	+0.693	−0.840	+0.601	**D**
4	**D**	+0.805	−0.693	+0.636	+0.693	+0.840	+2.281	**D**
5	**D**	−0.805	+0.693	+0.636	−0.693	+0.840	+0.672	**D**
6	**D**	−0.805	+0.693	+0.636	−0.693	+0.840	+0.672	**D**
7	**C**	−0.805	−0.693	+0.636	−0.693	+0.840	−0.715	**C**
8	**C**	−0.805	−0.693	+0.636	−0.693	−0.840	−2.394	**C**
9	**C**	−0.805	−0.693	+0.636	−0.693	−0.840	−2.394	**C**
10	**C**	−0.805	+0.693	+0.636	−0.693	−0.840	−1.008	**C**
11	**C**	−0.805	−0.693	−0.636	−0.693	+0.840	−1.988	**C**
12	**C**	−0.805	−0.693	−0.636	−0.693	−0.840	−3.667	**C**
A	?	−0.805	+0.693	+0.636	−0.693	+0.840	+0.672	**D**
B	?	+0.805	−0.693	+0.636	+0.693	−0.840	+0.601	**D**

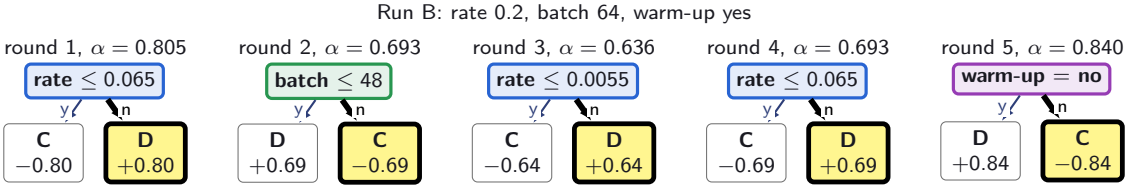
Entry $= \alpha_t\, h_t(x)$: plus the stump's vote weight if its leaf says **D**, minus if **C**. F_5 is the row sum. All twelve training runs are classified correctly. New runs: A $F_5 = 0.672$, B $F_5 = 0.601$.

Run A: the leaf reached in each stump



Yellow leaf = where A ends; the number is its vote $\alpha_t h_t$. Votes: -0.805, $+0.693$, $+0.636$, -0.693, $+0.840$, total $F_5 = 0.672$, so A is predicted **D**by a small margin: the stumps disagree.

Run B: the leaf reached in each stump



Votes: $+0.805$, -0.693, $+0.636$, $+0.693$, -0.840, total $F_5 = 0.601$: predicted **D**, again by a small margin. AdaBoost gives a class and a margin; it does not give a probability directly.

Same recipe, different loss

Part 3 reduced the **log loss** step by step. AdaBoost reduces the **exponential loss** $L = e^{-yF}$ for $y = \pm 1$ in exactly the same stagewise way. The derivative with respect to the score is $\dfrac{dL}{dF} = -y\, e^{-yF}$, so the number that plays the role of the residual for run i is

$$\text{(its size) } e^{-y_i F_i} \;=\; \text{the run's weight, before dividing by the total.}$$

Check after two rounds, $F = \alpha_1 h_1 + \alpha_2 h_2$:

run	1	2	3	4	5	6	7	8	9	10	11	12
F_2	+0.11	+1.50	+0.11	+0.11	−0.11	−0.11	−1.50	−1.50	−1.50	−0.11	−1.50	−1.50
e^{-yF_2}	0.894	0.224	0.894	0.894	1.118	1.118	0.224	0.224	0.224	0.894	0.224	0.224
÷7.16	0.1250	0.0312	0.1250	0.1250	0.1562	0.1562	0.0312	0.0312	0.0312	0.1250	0.0312	0.0312
update	0.1250	0.0312	0.1250	0.1250	0.1563	0.1563	0.0312	0.0312	0.0312	0.1250	0.0312	0.0312

The last two rows agree: the AdaBoost weights are the exponential-loss residuals, rescaled to add to 1. A mistake ($yF < 0$) makes $e^{-yF} > 1$ and so a large weight.

AdaBoost: summary of the procedure

step	what you do	which number is used
start	$w_i = \frac{1}{12}$, labels $y_i = \pm 1$	none
(a) stump	smallest $\varepsilon = \sum_{\text{mistakes}} w_i$; each leaf predicts its weighted majority	the weights of the mistakes
(b) vote weight	$\alpha = \frac{1}{2} \ln \frac{1-\varepsilon}{\varepsilon}$	ε of this stump
(c) new weights	$w_i \leftarrow w_i e^{-\alpha y_i h(x_i)} / Z$	α of this stump
predict	$F(x) = \sum_t \alpha_t h_t(x)$; **D** if $F > 0$	one signed vote per stump (the table of contributions)
stop	when held-out error stops falling	–

Part 5: XGBoost

parent node | yes-group | no-group | setting λ, γ | leaf weight w

What XGBoost changes

Same loop as Part 3: compute each run's current probability, fit a tree to what is left, add a shrunken correction. With $g_i = p_i - y_i = -r_i$ (slope), $h_i = p_i(1 - p_i)$ (curvature), and sums $G = \sum g_i$, $H = \sum h_i$ over a node:

	gradient boosting (Part 3)	XGBoost
question chosen by	largest $n_L \bar{r}_L^2 + n_R \bar{r}_R^2$	largest gain, built from $G^2/(H + \lambda)$ of the yes, no and parent nodes, minus γ (formula on a later slide)
leaf value	$\sum r / \sum p(1-p) = -G/H$	$w = -G/(H + \lambda)$
penalty on leaf values	none	λ: larger λ gives smaller $\lvert w \rvert$
penalty on tree size	none	γ: a split must gain more than γ
tree	a stump	deeper, while a split gains more than γ
update	$F \leftarrow F + \nu\gamma_{\text{leaf}}$	$F \leftarrow F + \nu w_{\text{leaf}}$

Symbol warning: Part 3 called the leaf value γ. XGBoost calls it w and uses γ for the penalty per leaf.

The objective and the leaf weight

For a new tree with T leaves and leaf weights $w_1, \dots, w_T$ (run i receives $w_{j(i)}$), XGBoost minimises, to second order in the correction,

$$\text{Obj} = \sum_i \left[g_i w_{j(i)} + \tfrac{1}{2} h_i w_{j(i)}^2 \right] + \gamma T + \tfrac{1}{2} \lambda \sum_j w_j^2 = \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T,$$

where G_j, H_j add g_i, h_i over the runs in leaf j. Each leaf is a separate quadratic in w_j. Set its derivative to zero:

$$G_j + (H_j + \lambda) w_j = 0 \implies \boxed{w_j = -\frac{G_j}{H_j + \lambda}}, \qquad \text{leaf contributes } -\tfrac{1}{2} \frac{G_j^2}{H_j + \lambda}.$$

Check against Part 3. With $\lambda = 0$: $w = -G/H = \sum r / \sum p(1-p)$, the Newton leaf value already used. The new ingredient is λ in the denominator: it pulls every leaf value towards 0, and more strongly when the leaf is small (H small).

The gain of a split

A node with sums (G, H) is split into (G_L, H_L) and (G_R, H_R). The objective before is $-\frac{1}{2}\frac{G^2}{H+\lambda} + \gamma$ (one leaf); after, $-\frac{1}{2}\frac{G_L^2}{H_L+\lambda} - \frac{1}{2}\frac{G_R^2}{H_R+\lambda} + 2\gamma$ (two leaves). The fall in the objective is

$$\text{gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma$$

Rule: among all questions take the largest gain; if it is ≤ 0 do not split (make a leaf).

Compare with Part 1. The entropy gain also compared the parent (parent) with its two groups (yes , no). Here the "impurity" of a group is $G^2/(H+\lambda)$: large when the runs in the group all push the same way (their g_i share a sign), zero when they cancel.

Round 1: gradients and curvatures

Settings used from here on: $\lambda = 1$, $\gamma = 0.25$, step size $\nu = 0.5$, trees at most two questions deep. All scores start at 0, so every $p = 0.5$.

runs	y	$g = p - y$	$h = p(1-p)$	how many
1–6 (**D**)	1	-0.5	0.25	6
7–12 (**C**)	0	$+0.5$	0.25	6

Whole data set, the parent node: $G = 6(-0.5) + 6(0.5) = 0$, $H = 12 \times 0.25 = 3$, so

$$\frac{G^2}{H+\lambda} = \frac{0^2}{3+1} = 0 .$$

Round 1: choose the root question

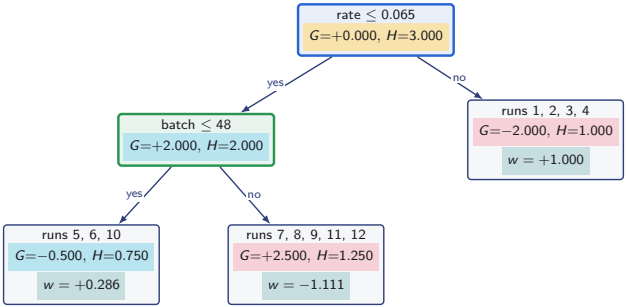
Every candidate question, with (G, H) of each group:

question	yes-group (G, H)	no-group (G, H)	gain (before γ)
rate ≤ 0.065	$+2.00,\ 2.00$	$-2.00,\ 1.00$	1.667
rate ≤ 0.02	$+1.50,\ 1.25$	$-1.50,\ 1.75$	0.909
rate ≤ 0.0055	$+1.00,\ 0.50$	$-1.00,\ 2.50$	0.476
rate ≤ 0.2	$+1.00,\ 2.50$	$-1.00,\ 0.50$	0.476
batch ≤ 48	$-1.00,\ 1.00$	$+1.00,\ 2.00$	0.417

Winner: rate ≤ 0.065.

$$\text{gain} = \tfrac{1}{2}\left[\frac{2^2}{2+1} + \frac{(-2)^2}{1+1} - \frac{0^2}{3+1}\right] = \tfrac{1}{2}\left[1.333 + 2.000 - 0\right] = \mathbf{1.667} > \gamma = 0.25$$

Round 1: grow the second level



Yes-group (runs 5–12), parent now $G = 2, H = 2$**.** Best question batch ≤ 48: gain $0.794 > \gamma$, so split. Next best: warm-up, 0.333.

No-group (runs 1–4), parent $G = -2, H = 1$**.** The best gain is -0.257 (negative: the four runs already agree, and splitting costs λ). No split: a leaf.

$\gamma = 0.25$ only needs to be below 0.794 for the second split to stay; $\gamma = 1$ would remove it.

Round 1: leaf weights

$$w_j = -\frac{G_j}{H_j + \lambda}, \qquad \lambda = 1$$

leaf	G	H	w with $\lambda = 1$	w with $\lambda = 0$
runs 5, 6, 10	-0.500	0.750	$+0.286$	$+0.667$
runs 7, 8, 9, 11, 12	$+2.500$	1.250	-1.111	-2.000
runs 1, 2, 3, 4	-2.000	1.000	$+1.000$	$+2.000$

Every leaf value is smaller than the Part 3 (Newton) value, by the factor $H/(H+\lambda)$. The leaf with the fewest runs and the least curvature (runs 5, 6, 10, $H = 0.75$) is shrunk the most: 0.286 against 0.667. Update: $F \leftarrow F + \nu w$ with $\nu = 0.5$.

Your turn: is this split worth it?

Work on this, then compare with the next slide

A node has $G = +1.0$, $H = 2.0$. A question splits it into a yes-group with $G_L = -1.0$, $H_L = 1.0$ and a no-group with $G_R = +2.0$, $H_R = 1.0$. Use $\lambda = 1$.

1. The gain (before γ)?
2. The leaf weights w_L and w_R if the split is kept?
3. Is the split kept when $\gamma = 0.25$? When $\gamma = 1.2$?

(About 3 minutes.)

Answer: is this split worth it?

1. gain $= \frac{1}{2}\left[\frac{(-1)^2}{1+1} + \frac{2^2}{1+1} - \frac{1^2}{2+1}\right] = \frac{1}{2}[0.5 + 2 - 0.333] = \mathbf{1.083}$.

2. $w_L = -\dfrac{-1}{1+1} = +0.5$, $\quad w_R = -\dfrac{2}{1+1} = -1.0$. (Without the split the node's single weight would be $-\frac{1}{3}$.)

3. Gain after the penalty: $1.083 - 0.25 = 0.833 > 0$: keep. With $\gamma = 1.2$: $1.083 - 1.2 < 0$: do not split, the node stays one leaf. A larger γ gives smaller trees.

Round 1: update every run

run	out	p before	g	leaf (runs)	w	νw	F after	p after
1	**D**	0.500	-0.500	(1, 2, 3, 4)	$+1.000$	$+0.500$	$+0.500$	0.622
2	**D**	0.500	-0.500	(1, 2, 3, 4)	$+1.000$	$+0.500$	$+0.500$	0.622
3	**D**	0.500	-0.500	(1, 2, 3, 4)	$+1.000$	$+0.500$	$+0.500$	0.622
4	**D**	0.500	-0.500	(1, 2, 3, 4)	$+1.000$	$+0.500$	$+0.500$	0.622
5	**D**	0.500	-0.500	(5, 6, 10)	$+0.286$	$+0.143$	$+0.143$	0.536
6	**D**	0.500	-0.500	(5, 6, 10)	$+0.286$	$+0.143$	$+0.143$	0.536
7	**C**	0.500	$+0.500$	(7, 8, 9, 11, 12)	-1.111	-0.556	-0.556	0.365
8	**C**	0.500	$+0.500$	(7, 8, 9, 11, 12)	-1.111	-0.556	-0.556	0.365
9	**C**	0.500	$+0.500$	(7, 8, 9, 11, 12)	-1.111	-0.556	-0.556	0.365
10	**C**	0.500	$+0.500$	(5, 6, 10)	$+0.286$	$+0.143$	$+0.143$	0.536
11	**C**	0.500	$+0.500$	(7, 8, 9, 11, 12)	-1.111	-0.556	-0.556	0.365
12	**C**	0.500	$+0.500$	(7, 8, 9, 11, 12)	-1.111	-0.556	-0.556	0.365

Log loss $0.693 \to 0.515$ (Part 3 stump: 0.504). Only run 10 is wrong ($p = 0.536$, it is **C**): it shares a leaf with runs 5 and 6.

Rounds 1 to 6

round	questions asked (root, then yes-side)	gains before γ	leaf weights w	loss	wrong
1	rate ≤ 0.065, then batch ≤ 48	1.667, 0.794	$+0.29$, -1.11, $+1.00$	0.515	10
2	rate ≤ 0.065, then batch ≤ 48	0.939, 0.462	$+0.23$, -0.84, $+0.78$	0.412	10
3	rate ≤ 0.065, then warm-up $=$ no	0.575, 0.343	$+0.17$, -0.75, $+0.64$	0.344	none
4	rate ≤ 0.065, then batch ≤ 48	0.379, 0.282	$+0.21$, -0.63, $+0.54$	0.295	none
5	rate ≤ 0.065	0.256	-0.26, $+0.46$	0.276	none
6	no split	all below γ	$+0.052$	0.276	none

Training error is 0 from round 3. In round 5 only the root clears γ; from round 6 on no question gains more than 0.25, so the tree is a single leaf that shifts every score by only 0.03, 0.02, 0.01 and the loss stays near 0.275.

What λ and γ do

Same twelve runs, two-question trees, $\nu = 0.5$, eight rounds:

settings	first round with no wrong run	wrong after round 8	log loss after 8	leaves in trees 1 to 8
$\lambda = 0,\ \gamma = 0$	2	none	0.049	3, 4, 4, 4, 4, 4, 4, 4
$\lambda = 1,\ \gamma = 0$	3	none	0.190	3, 3, 3, 3, 3, 3, 4, 3
$\lambda = 1,\ \gamma = 0.25$	3	none	0.275	3, 3, 3, 3, 2, 1, 1, 1
$\lambda = 1,\ \gamma = 0.5$	never	10	0.369	3, 2, 3, 1, 1, 1, 1, 1
$\lambda = 1,\ \gamma = 1$	never	5, 6	0.573	2, 1, 1, 1, 1, 1, 1, 1

Larger λ and γ give smaller leaf values and smaller trees: the training loss falls more slowly (or stops), which is the intended price of a simpler model. Too large a γ (≥ 0.5 here) stops learning before the training data are fitted; choose both on held-out data.

Runs A and B through the trees

round	run A (0.05, 32, no): path	νw	run B (0.2, 64, yes): path	νw
1	rate $\leq$ 0.065 yes; batch $\leq$ 48 yes	$+0.143$	rate $\leq$ 0.065 no	$+0.500$
2	rate $\leq$ 0.065 yes; batch $\leq$ 48 yes	$+0.113$	rate $\leq$ 0.065 no	$+0.389$
3	rate $\leq$ 0.065 yes; warm-up $=$ no yes	$+0.086$	rate $\leq$ 0.065 no	$+0.319$
4	rate $\leq$ 0.065 yes; batch $\leq$ 48 yes	$+0.104$	rate $\leq$ 0.065 no	$+0.269$
5	rate $\leq$ 0.065 yes	-0.131	rate $\leq$ 0.065 no	$+0.232$

Entry $= \nu w$ of the leaf reached. Rounds 6–8 add $+0.026$, $+0.017$, $+0.012$ to every run. Totals after 8 rounds: A $F = 0.369$, $p = 0.591$; B $F = 1.764$, $p = 0.854$. Both predicted **D**; B with more confidence. Unlike AdaBoost, XGBoost gives a probability.

XGBoost: summary of the procedure

step	what you do	which number is used
start	$F_i = 0$, $p_i = 0.5$; choose λ, γ, ν, depth	the three settings
(a) slopes	$g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$	each run's current p
(b) question	largest gain $\frac{1}{2}[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}] - \gamma$	G, H of the node, yes, no
(c) grow	split again while the gain exceeds 0; stop at depth limit	the gain after γ
(d) leaf weights	$w = -G/(H + \lambda)$	G, H of the leaf's runs and λ
(e) update	$F_i \leftarrow F_i + \nu w$	w of the leaf reached
predict	$F = \nu \sum_t w_t(\text{leaf})$, $p = S(F)$	one weight per tree

Part 6: Comparison

one tree, many trees averaged, many small trees added

Comparison of bagging and boosting

	bagging (random forest)	boosting
trees grown	side by side, each on its own bootstrap sample	one after another, each on the residuals so far
each tree	deep: low bias, high variance	small (a stump): high bias, low variance
combined by	average $\frac{1}{B}\sum_b T_b$	sum $F_0 + \nu \sum_t \gamma_t(\cdot)$
mainly reduces	variance, down to the floor $\rho\sigma^2$	bias, step by step
more trees	error settles; harmless	keeps fitting; stop using held-out data
main settings	number of trees; features per split	step size; rounds; depth

Gradient boosting, AdaBoost and XGBoost side by side

	gradient boosting (Part 3)	AdaBoost (Part 4)	XGBoost (Part 5)
each stump or tree is fitted to	residuals $y - p$	the data with run weights	slopes g and curvatures h
loss reduced	log loss	exponential loss e^{-yF}	log loss, plus penalties
question chosen by	$n_L \bar{r}_L^2 + n_R \bar{r}_R^2$	smallest weighted error ε	largest gain with $G^2/(H+\lambda)$
number each tree contributes	$\nu\gamma_{\text{leaf}}$	$\alpha\, h(x) = \pm\alpha$	νw_{leaf}
output	probability $S(F)$	class and margin F	probability $S(F)$
controls on size	step size, rounds	rounds	λ, γ, depth, step size

Predictions for the two new runs

method	run A (0.05, 32, no)	run B (0.2, 64, yes)
single tree	**D** (2/2 in its leaf)	**D** (4/4 in its leaf)
bagging, 3 trees	0 1 1 $\to \frac{2}{3}$, **D**	1 0 1 $\to \frac{2}{3}$, **D**
bagging, many trees (simulated)	≈ 0.872, **D**	≈ 0.934, **D**
boosting, 8 stumps	$F = 1.069$, $p = 0.744$, **D**	$F = 1.490$, $p = 0.816$, **D**
AdaBoost, 5 stumps	$F = 0.672$ (margin), **D**	$F = 0.601$ (margin), **D**
XGBoost, 8 trees	$F = 0.369$, $p = 0.591$, **D**	$F = 1.764$, $p = 0.854$, **D**

The single tree gives only a class; bagging shows how much the trees disagree (with many trees, about 0.9 of them say **D**); boosting gives a probability built from summed corrections.

Review questions

1. Gain of "batch ≤ 48" at the root? $1 - \frac{4}{12}(0.8113) - \frac{8}{12}(0.9544) = 0.0933$.
2. Which number do we read from tree 2 for run B, and why? $T_2(B) = 0$*: warm-up is yes, so it reaches the leaf of 4 draws, all* ***C****.*
3. $\sigma^2 = 1$, $\rho = 0.3$, $B = 20$: variance of the average, and the floor? $0.3 + 0.7/20 = 0.335$*; floor* 0.3.
4. Why does the variance table match σ^2/B but show no floor? *Only resampling varies; the floor comes from the dataset varying.*
5. In boosting round 1, why is $\gamma_{\text{no}} = +2$ but each score rises by 1? γ *is the full Newton step; the update multiplies by* $\nu = 0.5$.
6. After round 3 two runs are wrong though the loss fell: contradiction? *No: each round lowers the loss, not the error count.*

Review questions: AdaBoost and XGBoost

1. A stump has weighted error 0.2. Its vote weight? $\frac{1}{2}\ln 4 = 0.693$.
2. After the update, what is the weighted error of that same stump? *Exactly* $\frac{1}{2}$*, so it would get vote weight* 0.
3. Why are the AdaBoost run weights called exponential-loss residuals? $w_i \propto e^{-y_i F_i} = |dL/dF|$ *for* $L = e^{-yF}$.
4. What does λ do to a leaf value, and what does γ do to a tree? λ *shrinks* $w = -G/(H+\lambda)$*;* γ *blocks splits whose gain is below* γ.
5. With $\lambda = 0$, $\gamma = 0$ and stumps, which part of XGBoost still differs from Part 3? *The question score:* G^2/H *instead of* $n\bar{r}^2$.